Text and Sequence Analytics

Compiled Revision Notes

Table of Contents

Week 1: Introduction to Text & Sequence Analytics **5**
Fundamentals of Text Analytics . . . 5
The DIKW Pyramid . . . 5
Artificial Intelligence: Capabilities & Branches . . . 5
Sequence Analysis & Bioinformatics . . . 6
Historical Context & Thought Experiments . . . 6
Modern Tools & Frameworks . . . 6

Week 2: Text Processing & Normalization **6**
Text Representation . . . 6
Regular Expressions (Regex) . . . 7
Text Normalization . . . 7
Key Stemming Algorithms . . . 8
Part of Speech (POS) Tagging . . . 8
Performance Metrics & Data Structures . . . 8

Week 3: Tokenization & Sequence Segmentation **8**
Text Tokenization Basics . . . 8
Sub-Word Tokenization . . . 9
Sequence Segmentation . . . 10
SentencePiece Framework . . . 11

Week 4: Sequence Similarity & Alignment **12**
Mathematical Fundamentals of Similarity . . . 12
Distance Calculations . . . 12
Sequence Alignment . . . 13
Advanced Scoring Parameters . . . 14
BLAST Statistical Significance . . . 14

Week 5: Alignment-Free Sequence Similarity **15**
Fundamentals of Alignment-Free Similarity . . . 15
Measures vs. Metrics . . . 15
Indices, Coefficients, and Distances . . . 15
Vector-Based Distances . . . 15
Set-Based Similarities . . . 17
MinHash Algorithm . . . 19

Week 6: Text Feature Engineering **19**
Feature Engineering Fundamentals . . . 19
Challenges of Using Text . . . 19
Data Sets and Data Frames . . . 20
Text Encoding and Vectorization . . . 20
Term Frequency-Inverse Document Frequency . . . 20
Okapi BM25 . . . 21

Feature Selection and Hashing . . . 22
Dimensionality Reduction . . . 22

Week 7: Introduction to Text Classification **22**
Fundamentals of Text Classification . . . 22
Supervised Classification Models . . . 23
Neural Networks (NN) . . . 24
Activation Functions . . . 24
Data Normalization . . . 25
Gradient Descent & Accelerated Learning . . . 25
Network Topology . . . 26
Unsupervised Learning (Clustering) . . . 26

Week 8: Introduction to Word Embeddings **27**
Fundamentals of Word Embeddings . . . 27
Static vs. Context-Sensitive Embeddings . . . 27
Geometric Properties & Word Analogies . . . 27
One-Hot Encoding vs. Dense Embeddings . . . 28
Word2Vec Architecture . . . 28

Week 9: Word2Vec Methods & Skip-Gram vs. CBOW **29**
Continuous Bag of Words (CBOW) . . . 29
Skip-Gram . . . 29
Mathematical Objective: Softmax and Cross-Entropy . . . 30
Training Optimizations . . . 30
Skip-Gram vs. CBOW . . . 31

Week 10: Continued Word Embeddings & Transformer Architecture **32**
Doc2Vec . . . 32
Word2Vec Limitations . . . 32
Machine Learning with Embeddings . . . 32
Average Pooling . . . 33
Max Pooling . . . 33
Transformer Architecture . . . 34
What is Attention? . . . 34
Attention Scores . . . 34
Transformer Model - Preprocess & Input Embeddings . . . 34
Positional Encoding . . . 35
Transformer Model - Encoder . . . 35
Self-Attention . . . 35
Scaled Dot Product Attention . . . 35
Multi-head Attention . . . 36
Add & Normalize Layer . . . 37
Feed-Forward Network . . . 37
Encoder Output . . . 38
Transformer Model - Decoder . . . 38
Masked Multi-Head Attention . . . 38
Cross Attention . . . 39

Linear Layer & Softmax . . . 39

Week 11: Information Retrieval & Text Search **40**
Information Retrieval (IR) . . . 40
Inverted Index Data Structure . . . 40
Inverted Index Query Operations . . . 41
Inverted Index Optimisation . . . 41
Compressed Inverted Index . . . 42
Spam and Search Engines . . . 42
Link-Based Search . . . 43
Early Search Engines . . . 43
Hyperlink-Induced Topic Search (HITS) . . . 43
Limitations of HITs . . . 44
PageRank . . . 44
TrustRank . . . 45

Week 12: Sentiment Analysis **45**
Sentiment Analysis Fundamentals . . . 45
Evaluation Methods . . . 45
Key Lexicons . . . 46
Scoring Mechanisms . . . 47
VADER Specific Properties . . . 47
Distance-Weighted Negation . . . 48

Week 1: Introduction to Text & Sequence Analytics

Fundamentals of Text Analytics

Text analytics, or **text mining**, is the process of extracting high-quality information from unstructured text. It differs from standard data processing because text is inherently 'noisy' and lacks a regular syntax or pattern.

- **Definition:** Transforming unstructured text into useful data through NLP, algorithms, and analytical methods.
- **The 3 Vs:** The vast amount of data created yearly (Volume, Velocity, Variety) necessitates modern storage like NoSQL and Vector Databases.
- **Core Tasks:**
  - Translation and Language Identification
  - Sentiment Analysis and Topic Modelling
  - Classification and Clustering

The DIKW Pyramid

This hierarchy illustrates how we move from raw facts to actionable insight. In exams, this is often used to explain the goal of information management vs. knowledge management.

Level	Description	Focus
Wisdom	Understanding 'Why'; used for high-level decision making	Know Why
Knowledge	Finding patterns, rules, and building predictive models	Know How
Information	Data that has been categorized, combined, and calculated	Know What
Data	Raw facts, figures, and measurements	Raw Input

Artificial Intelligence: Capabilities & Branches

AI is categorized by its 'reach' relative to human intelligence.

AI by Capability

- **Artificial Narrow Intelligence (ANI):** Specialized in one task (e.g., Alexa, Siri, or Generative AI).
- **Artificial General Intelligence (AGI):** Can perform any task a human can.
- **Artificial Super Intelligence (ASI):** Surpasses all human capabilities.

The Seven Branches (IEEE)

- **Machine Learning (ML):** Statistical models that learn from data.
- **Natural Language Processing (NLP):** Understanding and generating human language.
- **Neural Networks:** Models based on the human brain (interconnected nodes).
- **Computer Vision:** Interpreting visual or image data.
- **Robotics:** Programming machines for autonomous tasks.

- **Expert Systems:** Simulating human expertise in specific fields.
- **Fuzzy Logic:** Handling vagueness and uncertainty in reasoning.

Sequence Analysis & Bioinformatics

A unique aspect of this course is the crossover between human language and biological 'languages' like DNA.

- **Goal:** Analyzing DNA and protein sequences to discover biological insights.
- **The Connection:** Biological sequences use symbols (like English) where the order is the most critical factor, just like NLP.
- **Key Growth:** The volume of data in GenBank (Nucleotides, Amino Acids) has seen exponential growth over 30 years.

Historical Context & Thought Experiments

Dr. Healy highlights several milestones that defined the field:

- **Memex (1945):** Vannevar Bush's concept for 'associative memory' storage, the precursor to hypertext.
- **Turing Test (1950):** A test of whether a machine can 'imitate' a human well enough to fool an interrogator.
- **Searle's Chinese Room (1980):** A critique of 'Strong AI'. It argues that a machine can manipulate symbols (syntax) without ever understanding their meaning (semantics).
- **AI Winters:**
  - 1st AI Winter: Triggered by the 1966 ALPAC report.
  - 2nd AI Winter: Mid-1980s to early 1990s.

Modern Tools & Frameworks

To implement these theories, the industry uses specific Python and Java ecosystems.

- **General ML:** scikit-learn, PyTorch, TensorFlow, Keras.
- **NLP Specific:** NLTK, spaCy, Gensim.
- **LLM Ecosystem:** Hugging Face, LangChain, and models like GPT, Gemini, and Llama.
- **Data Interchange:** ONNX (Open Neural Network Exchange) allows models to move between different languages like Python and Java.

Week 2: Text Processing & Normalization

Text Representation

Computers do not process characters directly; they represent them as numeric values.

- **ASCII:** A basic numeric mapping for 128 characters, including control characters (like 'Line Feed') and standard Latin letters.
- **ISO-8859-1 (Latin 1):** An extension of ASCII standardized in 1987 to include 256 characters, covering most Latin and Greek alphabets.
- **Unicode:** The modern standard for world languages.
  - **UTF-8:** Uses 1 byte for basic characters; compatible with ISO-8859-1.
  - **UTF-16:** Uses 2 bytes (16 bits) per character, allowing for 65,536 characters, including ancient scripts like Ogham.

Java Strings

- Java uses UTF-16 internally for `char` and `String`.
- **Immutability:** Once a `String` is declared, it cannot be changed; modifying it actually creates a new object to improve performance.
- **String Pooling:** Literal strings (e.g., `String t = "Happy Days!"`) are shared in a 'String Pool' in the JVM heap to save memory.

Regular Expressions (Regex)

Developed in the 1950s, Regex is the 'de facto' standard for pattern matching and text manipulation.

Symbol	Meaning	Example
`.`	Any single character	`a.b` → `acb`
`\d`	Any digit (0-9)	`\d{4}` → `1970`
`\w`	Word character (a-z, 0-9)	`\w+` → `Hello123`
`^` / `$`	Start / End of string	`^Hi` / `end$`
`[^a-zA-Z]`	Match only letters	Useful for removing numbers or symbols

---

Text Normalization

Before analysis, text must be 'wrangled' or cleaned through a series of steps:

- **Tokenization:** Decomposing text into minimal meaningful units like words or sentences.
- **Morpheme Analysis:** Breaking words into stems and affixes (prefixes or suffixes) to find the 'base' meaning.
- **Stemming:** A heuristic 'affix stripper' that reduces a word to a root.
- **Lemmatization:** A more accurate, context-aware method that uses a dictionary (like WordNet) to find the valid base form (lemma).
- **Stop Word Removal:** Filtering out noisy, insignificant words (like 'the', 'and') that often make up 25%+ of a document.

Key Stemming Algorithms

Stemming is a trade-off between speed and accuracy.

- **Porter's Algorithm (1980):** The most widely used; classifies characters as Vowels (v) or Consonants (c) and applies a 5-step rule list.
- **Snowball Stemmer:** An updated version of Porter's algorithm supporting multiple languages.
- **Lancaster (Paice-Husk) Stemmer:** A 'greedy' iterative rules-based approach.

Common Issues:

- **Over-stemming:** Too aggressive; stems 'university' to 'universe'.
- **Under-stemming:** Too lazy; fails to stem 'knavish' to 'knave'.

Part of Speech (POS) Tagging

POS tagging labels words with lexical classes (noun, verb, etc.) based on their role in a sentence.

- **Penn Treebank:** The standard notation for tagging.
- **Common Tags:**
  - `NN`: Noun, singular
  - `VB`: Verb, base form
  - `JJ`: Adjective
  - `CD`: Cardinal number
  - `PRP`: Personal pronoun

Performance Metrics & Data Structures

- **Bloom Filter:** A probabilistic data structure used to test set membership efficiently; ideal for representing large sets of stop words or computing similarity metrics while saving memory.
- **Index Compression Factor (ICF):** Measures a stemmer's strength by the percentage of distinct words it reduces.

$$\text{ICF} = \frac{n - s}{n} \times 100$$

where n is the number of distinct words before stemming and s is the number after stemming.

Week 3: Tokenization & Sequence Segmentation

Text Tokenization Basics

- **Definition**: Segmenting text into meaningful atomic units (tokens) such as characters, sub-words, or words.

- **Vocabulary**: The complete set of unique tokens used by a specific tokenizer.

- **Out-of-Vocabulary (OOV)**: Words appearing in text that are missing from the tokenizer's pre-defined vocabulary.

Granularity Trade-offs

- **Word-level**: Often used for semantic analysis and Part-of-Speech tagging, but results in large vocabularies and frequent OOV misses.

- **Character-level**: Useful for spelling correction and OOV handling, but creates long sequences that are computationally expensive to process.

Sub-Word Tokenization

Decomposes words into smaller units to handle OOV words and provide better morphological understanding while keeping vocabulary size manageable.

Byte-Pair Encoding (BPE)

- **Concept**: A data compression algorithm adapted for tokenization by iteratively merging the most frequent adjacent pairs.

- **Models**: Used in GPT-2, RoBERTa, and XLM.

BPE Training Algorithm

1. Initialize vocabulary: $V = \text{Set of characters in } S$
2. **Do**:
   - Find: $(\text{Token}_{Left}, \text{Token}_{Right}) = \text{getMostFrequentAdjacentTokens}(S)$
   - Concatenate: $\text{Token}_{new} = \text{Token}_{Left} + \text{Token}_{Right}$
   - Update: $V = V + \text{Token}_{new}$
   - Replace all adjacent $(\text{Token}_{Left}, \text{Token}_{Right})$ in S with Token_{new}.
3. **Loop Until** `maxMerges` reached.

- **Complexity**:
  - Training time: $O(VC)$ where V is vocabulary size and C is corpus size.
  - Tokenization for text T: $O(T \log V)$

WordPiece

- **Concept**: Iterative sub-word tokenization similar to BPE but uses a likelihood scoring system for merges.

- **Models**: Primarily used by BERT.

Merging Score Formula

$$\text{score} = \frac{\text{freq}(\text{Token}_{Left}) \times \text{freq}(\text{Token}_{Right})}{\text{freq}(\text{Token}_{new})}$$

Markers

- Uses `##` to track sub-word positions (start, middle, or end) and preserve morphological structure.

Special Tokens

- `[CLS]`: Classification token at the start of every sequence.
- `[SEP]`: Separator for different text segments (e.g., Q&A).
- `[UNK]`: Unknown token for OOV words.
- `[PAD]`: Padding to reach fixed-size vector lengths.
- `[MASK]`: Used in BERT pre-training to hide words for prediction.

Unigram Tokenization

- **Concept**: A probabilistic algorithm that starts with a large vocabulary and iteratively removes tokens that cause the least log-likelihood loss.
- **Base Rule**: Base characters are never removed to ensure all strings remain tokenizable.

Log Loss Formula

$$\text{loss} = \sum_{i=1}^{n} \log \left(\sum_{w \in S(w_i)} p(w) \right)$$

Where $S(w_i)$ is the set of all possible tokenizations of a word w_i.

- **Training Complexity:** $O(nv)$ where n is training examples and v is vocabulary size.

Sequence Segmentation

Division of already tokenized text into smaller, overlapping sub-sequences, often using a sliding window.

N-grams

- **Definition**: A sequence of n contiguous characters.

- **Applications**: Spell checking, language modeling, and plagiarism detection.
- **Biological Context**: Called **k-mers** when used with DNA or protein sequences.
- **DNA Encoding**:
  - DNA sequences ($|\Sigma| = 4$) can be encoded with 2 bits:
    - A: 00
    - T: 01
    - G: 10
    - C: 11
  - A 64-bit long can encode a 32-mer.

Shingles

- **Definition**: A word or group of words, similar to n-grams but typically used for document-level analysis.
- **Applications**: Near-duplicate detection, document clustering, and data deduplication.

Skip-grams

- **Definition**: Similar to n-grams but allows gaps (skips) between words.
- **Value**: Captures semantic relationships between non-adjacent words, essential for creating dense word embeddings.

SentencePiece Framework

- **Origin**: Developed by Meta.

Features

- Implements optimized BPE, WordPiece, and Unigram (default).
- Reversible tokenization: (token id) $\leftrightarrow$ (Unicode)
- Excellent for languages without whitespaces (Chinese/Japanese) because it treats the whole sentence as a Unicode stream.
- Training time is reduced to $O(n \log n)$ using an **Enhanced Suffix Array (ESA)**.

Week 4: Sequence Similarity & Alignment

Mathematical Fundamentals of Similarity

To compare sequences, we distinguish between measuring how much they are alike versus how much they differ.

Similarity (s)

Measures the degree of correspondence between sequences. Higher similarity implies a closer relationship.

Distance (d)

Measures the number of changes required to transform one sequence into another.

Identity (I)

A Boolean indicator function:

$$I(a,b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{if } a \neq b \end{cases}$$

Metric Properties

For a distance function $d(x,y)$ to be a true metric, it must satisfy:

1. **Non-negativity:** $d(x,y) \geq 0$
2. **Identity of Indiscernibles:** $d(x,y) = 0 \iff x = y$
3. **Symmetry:** $d(x,y) = d(y,x)$
4. **Triangle Inequality:** $d(x,z) \leq d(x,y) + d(y,z)$

Distance Calculations

These quantify the cost of transforming one string into another.

Hamming Distance

The number of positions at which corresponding symbols differ.

Constraint: Sequences must be of equal length.

$$d_H(s_1, s_2) = \sum_{i=1}^{n} \mathbf{1}[s_{1i} \neq s_{2i}]$$

where $\mathbf{1}[\cdot]$ is the indicator function.

Levenshtein (Edit) Distance

The minimum number of insertions, deletions, or substitutions required to transform string A into string B.

Let $d(i, j)$ denote the distance between the first i characters of A and the first j characters of B.

Recurrence Relation:

$$d(i,j) = \min \begin{cases} d(i-1,j)+1 & \text{(Deletion)} \\ d(i,j-1)+1 & \text{(Insertion)} \\ d(i-1,j-1)+\text{cost} & \text{(Substitution)} \end{cases}$$

$$\text{where cost} = \begin{cases} 0 & \text{if } A[i] = B[j] \\ 1 & \text{if } A[i] \neq B[j] \end{cases}$$

Sequence Alignment

Alignment algorithms use a scoring matrix H and defined transition rules to find an optimal alignment.

Needleman-Wunsch (Global Alignment)

Aligns sequences from beginning to end. Suitable for closely related sequences of similar length.

Initialization: $H(i, 0) = i \times \text{gap}$ and $H(0, j) = j \times \text{gap}$

Recurrence Relation:

$$H(i,j) = \max \begin{cases} H(i-1,j-1) + S(a_i, b_j) & \text{(Match/Mismatch)} \\ H(i-1,j) + \text{gap} & \text{(Gap in B)} \\ H(i,j-1) + \text{gap} & \text{(Gap in A)} \end{cases}$$

Time complexity: $O(mn)$

Smith-Waterman (Local Alignment)

Finds the highest scoring local subsequences within two sequences.

Initialization: $H(i, 0) = 0$ and $H(0, j) = 0$

Recurrence Relation:

$$H(i,j) = \max \begin{cases} 0 & \text{(Restart)} \\ H(i-1,j-1) + S(a_i, b_j) & \text{(Match/Mismatch)} \\ H(i-1,j) + \text{gap} & \text{(Gap in B)} \\ H(i,j-1) + \text{gap} & \text{(Gap in A)} \end{cases}$$

The inclusion of 0 prevents negative scores and allows the alignment to restart at any position.

Advanced Scoring Parameters

Affine Gap Penalty

More realistic than a linear gap penalty because opening a gap is typically more costly than extending one.

$$W = g + (L-1)e$$

where:

- g = gap opening penalty
- e = gap extension penalty
- L = total length of the gap

BLAST Statistical Significance

BLAST evaluates whether an alignment is statistically meaningful.

Bit Score S'

A normalized alignment score independent of database size.

E-value (Expect Value)

The expected number of matches occurring by chance.

$$E = m \times n \times 2^{-S'}$$

where:

- m = query length
- n = database length
- S' = bit score

A very small or zero E-value indicates a statistically significant match.

Week 5: Alignment-Free Sequence Similarity

Fundamentals of Alignment-Free Similarity

Compare strings by token composition, ignoring positional information. Strings are represented as sets or bags of tokens (words, symbols, n-grams) rather than ordered sequences.

- $O(n)$ vs $O(n^2)$ for DP/alignment-based methods.
- Use **hashing** to convert variable-length tokens into fixed-length integers for ML pipelines.
- Can produce false positives when strings share composition but not structure, as positional information is lost.
- Token size (n-gram size) affects sensitivity: larger n-grams are more precise but sparser.
- Applications: plagiarism detection, text classification, clustering, genomics.

Measures vs. Metrics

A distance **measure** is not necessarily a **metric**. The four metric properties are covered in Week 4. The key point is that many alignment-free measures (e.g., Overlap coefficient) do not satisfy all four, so they are measures, not metrics.

Indices, Coefficients, and Distances

- **Coefficient:** A ratio-based scaling factor between sets.
- **Index:** A normalized similarity in $[0, 1]$, possibly combining multiple factors (e.g., Tversky Index of $0.8 = 80\%$ similar).
- **Distance:** Dissimilarity score where 0 implies identity.

Key Relationship: Similarity and distance are complementary. For any normalized similarity measure $\text{sim}(A, B)$, the corresponding distance is defined as:

$$d(A, B) = 1 - \text{sim}(A, B)$$

This applies to Cosine, Jaccard, Dice, Tversky, and Overlap measures.

Vector-Based Distances

Text is converted into a high-dimensional vector space using character frequencies, n-gram counts, or word embeddings. BOW structures ensure uniform-length vectors.

Cosine Similarity and Distance

Cosine of the angle between two n-dimensional vectors. Independent of magnitude. Used in transformers, clustering, and bioinformatics.

- **Cosine Similarity:**

$$\cos(s,t) = \frac{\sum_{i \in V} s_i t_i}{\sqrt{\sum_{i \in V} s_i^2}\sqrt{\sum_{i \in V} t_i^2}}$$

- **Cosine Distance:**

$$d_{cos}(s,t) = 1 - \cos(s,t)$$

The similarity value is bounded between 0 and 1: - $\cos = 0$ (90°): vectors are orthogonal (maximally dissimilar). - $\cos = 1$ (0°): vectors are parallel (maximally similar).

Token size matters. For two sentences with the same words in a different order:

Token Type	$\cos(A, B)$	Why
1-grams	≈ 1.0	Same word counts, order ignored
3-grams (word)	$= 0$	Phrases differ entirely
5-mers (char)	$= 0$	Character windows differ entirely

Euclidean Distance

Straight-line distance between two points. Sensitive to scale and has no notion of ordering: anagrams like *listen* and *silent* have identical character frequencies, so $d_E = 0$ despite different meanings.

$$d_E(s,t) = \sqrt{\sum_{i=1}^{n} (s_i - t_i)^2}$$

Manhattan Distance

Also called L_1, city block, or taxicab distance. Sums the absolute difference at each vector coordinate.

- **Manhattan Distance:**

$$d_M(s,t) = \sum_{i=1}^{n} |s_i - t_i|$$

Requires padding for strings of different lengths. Good when frequency differences matter more than order, e.g., spelling errors and anagram detection.

Canberra Distance

Weighted Manhattan where each difference is normalized by the sum of the values, making it less sensitive to scale. Range: $[0, n]$.

- **Canberra Distance:**

$$d_C(s,t) = \sum_{i \in V} \frac{|s_i - t_i|}{|s_i| + |t_i|}$$

Use when frequencies vary a lot in scale, or when small differences in small values are important. Prefer Manhattan when scale is consistent.

Chebyshev Distance

Also called L_∞ distance. Takes only the single largest difference between any coordinate, ignoring all others.

- **Chebyshev Distance:**

$$d_\infty(s,t) = \max_{i \in V} |s_i - t_i|$$

Use when the worst-case difference is what matters. Good for anomaly detection and KNN.

Minkowski Distance

Generalized distance that unifies Manhattan, Euclidean, and Chebyshev.

- **Minkowski Distance:**

$$d_p(s,t) = \left(\sum_{i \in V} |s_i - t_i|^p \right)^{\frac{1}{p}}$$

- $p = 1$ is equivalent to Manhattan distance.
- $p = 2$ is equivalent to Euclidean distance.
- $p = \infty$ is equivalent to Chebyshev distance.

Tune p to change sensitivity: higher p emphasizes the largest differences, at the cost of more computation.

Set-Based Similarities

These methods measure the similarity between sets of tokens derived from text.

Jaccard Similarity

Proposed by Paul Jaccard in 1901. Quantifies similarity by comparing shared elements against the total combined unique elements.

- **Jaccard Index:**

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

- **Jaccard Distance:**

$$d_J(A, B) = 1 - J(A, B)$$

Overlap Distance

Szymkiewicz-Simpson coefficient (1934/1960). Normalizes by the smaller set, making it useful for documents of very different sizes.

- **Overlap Coefficient:**

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

- **Overlap Distance:**

$$d_{\text{overlap}}(A, B) = 1 - \text{overlap}(A, B)$$

Sørensen-Dice Similarity

Weighted intersection, making it more sensitive to common elements. Smaller denominator penalizes mismatches less harshly than Jaccard. Better for short or variable-length texts.

- **Dice Index:**

$$\text{dice}(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

- **Dice Distance:**

$$d_{\text{dice}}(A, B) = 1 - \text{dice}(A, B)$$

Tversky Distance

An asymmetric similarity measure that quantifies the degree of overlap while explicitly weighting false positives and false negatives.

- **Tversky Index:**

$$\text{Tversky}(A, B) = \frac{|A \cap B|}{|A \cap B| + \alpha|A \setminus B| + \beta|B \setminus A|}$$

- **Tversky Distance:**

$$d_{\text{Tversky}}(A, B) = 1 - \text{Tversky}(A, B)$$

- Generalizes Jaccard when $\alpha = 1$ and $\beta = 1$.
- Generalizes Sørensen-Dice when $\alpha = 0.5$ and $\beta = 0.5$.

MinHash Algorithm

Developed by Andrei Broder (AltaVista, 1997). Represents each document as a signature of k minimum hash values from k different hash functions (using XOR and bit rotations). Scales Jaccard estimation to massive datasets.

- **MinHash Approximation:**

$$J(A, B) \approx \frac{n}{k}$$

- k is the total number of hash functions applied.
- n is the number of hash functions for which $h_{\min}(A) = h_{\min}(B)$.

The **expected value** of the MinHash similarity between two sets equals the Jaccard Index:

$$\mathbb{E}\left[\mathbf{1}[h_{\min}(A) = h_{\min}(B)]\right] = J(A, B)$$

MinHash Compatibility

Only works for set-based similarity measures.

Measure	Compatible?	Notes
Jaccard	Yes	Directly estimated
Sørensen-Dice	Yes (indirectly)	Derivable from Jaccard: $\text{Dice} = \frac{2J}{1+J}$ and $J = \frac{\text{Dice}}{2-\text{Dice}}$
Tversky	Yes (when $\alpha = \beta = 1$)	Reduces to Jaccard in this case
Overlap	No	Requires $\min(\lvert A\rvert, \lvert B\rvert)$, which MinHash cannot estimate
Cosine / Euclidean / Manhattan	No	These require vector arithmetic, which cannot be accurately estimated from hash signatures

Week 6: Text Feature Engineering

Feature Engineering Fundamentals

Before text can be processed by machine learning algorithms, it must be converted into a fixed-size input vector.

- Fixed-size inputs are ideal for continuous signals (e.g., sensors or game controllers), but text provides variable-length input.
- Encoding transforms variable-length text into a fixed format through mechanisms like n-grams, Huffman codes, or hashing.

Challenges of Using Text

- Context dependency dictates that meaning relies heavily on surrounding text and external knowledge.

- High-dimensional sparse vector spaces are generated by techniques like Bag of Words (BOW) or TF-IDF, leading to computational expense and the risk of overfitting.
- Linguistic nuances such as polysemy (multiple meanings for one word), synonymy (different words with similar meanings), and varying grammar rules complicate analysis.
- Extensive preprocessing is mandatory, including tokenization, lemmatization, stemming, and special character handling.

Data Sets and Data Frames

- A dataset is a general collection of data (structured, unstructured, or semi-structured).
- A data frame is a structured 2D matrix (rows + columns) used for ML, with defined data types.
- Within a data frame, continuous variables are typically normalized (rescaled between a minimum and maximum) or standardized (rescaled around a mean of zero and a standard deviation of one).
- Categorical or discrete variables are typically processed using one-hot encoding.

Text Encoding and Vectorization

Bag of Words (BOW)

- A multiset of tokens and their frequencies within a document.
- Can be extended to n-grams, shingles, or sub-words.
- **Limitation:** Loses word order $\rightarrow$ no contextual/semantic meaning.

Count Vectorization

- Creates a document-term matrix where each dimension corresponds to a specific token from the corpus.
- Values can represent raw frequencies, binary occurrences, or weighted measures.
- **Process:**
  1. Build a vocabulary (unique tokens across corpus)
  2. Create document-term matrix
  3. Encode each document as a vector

Shingles (n-grams)

- Help preserve local semantics and partial word order.

One-Hot Encoding

- Primarily used for categorical data rather than full text, this technique represents data as binary vectors where the vector length equals the total vocabulary size.

Term Frequency-Inverse Document Frequency

The TF-IDF model normalizes token frequencies by considering their relevance across the entire corpus, filtering out low-entropy "noise" words.

- **Term Frequency (TF):** Measures how frequently a term appears within a specific document relative to its length.

- **Term Frequency Formula:**

$$tf(t,d) = \frac{f(t,d)}{len(d)}$$

- **Inverse Document Frequency (IDF):** Measures the rarity and overall importance of a term across all documents.
- **Inverse Document Frequency Formula:**

$$idf(t,D) = \log\left(\frac{D}{n}\right)$$

- **TF-IDF Calculation:** The final weight is the product of TF and IDF, which highlights terms that are highly distinctive to a specific document.
- **TF-IDF Formula:**

$$tfidf(t,d,D) = \frac{f(t,d)}{len(d)} \times \log\left(\frac{D}{n}\right)$$

- TF-IDF reduces the weight of very common terms across documents (e.g., stop words) while increasing the relative importance of rarer, more informative tokens.

Okapi BM25

- A ranking function used in information retrieval systems (e.g. search engines)
- Balances:
  - Term frequency
  - Document length normalization
  - Term rarity (IDF)
- Short documents are boosted, long documents are penalized
- Long documents heavily penalize term weights in the denominator, while shorter documents increase the term weight.
- Parameter k_1 controls the scaling of term frequency, while b controls the strength of document length normalization.
- Computational complexity is historically $O(QC)$, but it is optimized in practice using inverted indices, document partitioning, and early termination heuristics.
- **BM25 Score:**

$$score(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

- **Okapi BM25 IDF:**

$$IDF(q_i) = \log\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right)$$

- **BM25 Optimizations:**

- Inverted indices (term → list of documents)
- Document partitioning (parallel processing)
- Early termination (stop when enough good results found)

Feature Selection and Hashing

Incremental Feature Selection

- A generate-and-test topology approach for neural networks to determine the optimal hidden layer configurations.
- **Hidden Layer Node Calculation:**

$$\text{Nodes} = \frac{|D|}{\alpha \times (N_{input} + N_{output})}$$

- $|D|$ = number of training samples, N_{input} = input neurons, N_{output} = output neurons, α = scaling factor (typically 2–10).

Vector Hashing

- Hashes variable-sized inputs (like n-grams) into a fixed-size vector array to drastically reduce vocabulary tracking.
- Converts variable-length input into fixed-length vectors
- Especially useful for text and high-dimensional data
- This is achieved by taking the modulus of the hash against the target array size.
- **Modulo Hash Index:** $index = hash(f)$ % n (where % is the *mod* operator)
- Particularly useful for domains with extremely large combinatorial feature spaces, such as biological sequences.
- For example, proteins built from 20 amino acids produce rapidly growing n-gram feature spaces:
  1-grams = 20, 2-grams = 400, 3-grams = 8,000, 4-grams = 160,000, 5-grams = 3,200,000.
- Hashing allows all such features to be compressed into a single fixed-length vector.

Dimensionality Reduction

To manage the sparsity and extreme dimensionality of text features, dimensionality reduction algorithms are commonly applied before classification.

- **Principal Component Analysis (PCA):** An unsupervised technique that projects data onto orthogonal axes to maximize variance.
- **Linear Discriminant Analysis (LDA):** A supervised technique that maximizes class separability.

Week 7: Introduction to Text Classification

Fundamentals of Text Classification

Text classification is the automated categorization of documents into predefined classes using machine learning models.

- **Applications:** Spam filtering, document organization, sentiment analysis, and topic modelling.
- **Model Types:** Classifiers can be supervised or unsupervised, and binary or multiclass.
- **Properties of Text Data:** Text is feature-rich and inherently high-dimensional.
- **Explainability (XAI):** Important but not always possible. Techniques like Shapley Values and LIME can help explain specific models, though some, like Neural Networks, remain black boxes.
- **Process Pipeline:**
  1. Preprocessing (cleaning and normalization).
  2. Feature extraction (BOW, TF-IDF, embeddings, hashing).
  3. Train/test split using a labeled corpus of documents.
  4. Model training and validation using standard ML statistical methods.
  5. Process, predict, and evaluate.

Supervised Classification Models

Different machine learning algorithms offer various trade-offs for text classification tasks.

Logistic Regression

A statistical model that predicts the probability of a binary outcome.

- **Advantages:** Highly interpretable (coefficients directly indicate feature importance), computationally efficient, and handles sparse high-dimensional data well.
- **Mechanism:** Applies the sigmoid function to a linear combination of features to output a probability between 0 and 1.
- **Logistic Function:**

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_n X_n)}}$$

Naive Bayes

A probabilistic classifier based on Bayes' theorem that assumes naive independence between features.

- **Advantages:** Very fast, simple, and serves as a strong computational baseline for text classification.
- **Mechanism:** Calculates probability based on word frequencies within a specific class. The independence assumption rarely holds true in natural language, but the model remains surprisingly effective.
- **Probability Equation:**

$$P(\text{doc}|\text{class}) = P(\text{word}_1|\text{class}) \times P(\text{word}_2|\text{class}) \times \cdots \times P(\text{word}_n|\text{class})$$

Support Vector Machines (SVM)

Developed in 1995, SVMs map data into a high-dimensional space to find an optimal separating hyperplane.

- **Mechanism:** Maximizes the margin between classes using support vectors (the specific data points closest to the hyperplane boundary).
- **Multiclass Handling:** Naturally a binary classifier, but handles multiclass environments using One-vs-Rest (OvR).
- **Advantages:** Highly effective for medium-sized datasets where a clear margin of separation exists, but computationally intensive for very large datasets.

Ensemble Methods & Random Forests

Combines multiple machine learning models into a single predictive model to improve overall accuracy, stability, and generalization capacity ("wisdom of crowds").

- **Mechanisms:**
  - **Bagging:** Reduces variance by training multiple models on random subsets of the training data with replacement.
  - **Boosting:** Reduces bias by building models sequentially, with each new model focusing on correcting the errors of the previous ones.
- **Random Forest:** An ensemble of decision trees where each tree is trained on a random data subset and makes an independent prediction. The forest's final prediction is determined by a majority vote. This significantly reduces the overfitting that individual trees are prone to.

Neural Networks (NN)

A mathematical model designed to simulate the structure and function of a biological neural network.

- **Structure:** Consists of highly interconnected layers of information-processing units called neurons.
- **Weights:** The knowledge of the network is stored in the weighted edges connecting nodes across layers. These weights are initialized randomly and updated during training.
- **Inputs and Outputs:**
  - All inputs must be converted into a fixed-sized feature vector before processing.
  - Each neuron receives multiple inputs, sums them, applies an activation function, and produces exactly one output.
  - A single neuron alone can only learn linearly separable tasks.

Activation Functions

Used by a neuron to compute its activation level based on inputs and weights. Activation functions should ideally be computationally simple (avoiding complex exponents) to maximize processing

speed during training iterations.

- **Primary Activation Functions:**
  - **Softmax:** Widely used in the output layer for multi-class classification. It transforms raw output scores (logits) into a probability distribution where all scores sum to 1. It pairs naturally with cross-entropy loss.
  - **Sigmoid:** A soft limiter operating smoothly within bounds of $[-4, 4]$ but optimally in $[-1, 1]$.
  - **Hyperbolic Tangent (Tanh):** A soft limiter that outputs values bounded between 0 and 1.
  - **ReLU (Rectified Linear Unit):** Computationally lightweight and heavily relied upon in deep learning.
  - **Leaky ReLU:** A variation of standard ReLU designed to prevent "dead" unrecoverable neurons.
- **Secondary Functions:** Step, Identity, LogSig, Gaussian, ELU, SELU, Softplus, Softsign, Swish, and Sinc.

Data Normalization

Normalizing data squashes inputs into a consistent, restricted range (typically $[-2, 2]$) to ensure network stability.

- **Importance:** Prevents extreme input values from immediately driving neuron outputs to absolute highs or lows, which is crucial for the stability of activation functions.
- **Application:** Normalization is strictly applied to the columns of a dataset, not the rows. It is generally unnecessary when using Tree-based ML models.
- **De-normalization:** Training a network on normalized data means it will produce normalized outputs, which must often be de-normalized for human interpretation.

Gradient Descent & Accelerated Learning

Gradient descent algorithms iteratively minimize training error by locating the lowest possible point on an error surface.

- **Learning Rate (α):**
  - Small α: Results in smaller weight adjustments, slower learning speeds, and a smooth, stable learning curve.
  - Large α: Accelerates learning drastically but risks inducing instability and oscillation within the network.
- **Momentum (β):** A parameter added to the delta rule to accelerate convergence, commonly set to $\beta \approx 0.95$.
- **Optimizers:** Categorized fundamentally into gradient-based systems (Momentum, Nesterov) and learning-rate-based systems (AdaGrad, RMSProp, Adam, AdaDelta).

Network Topology

The internal architecture of a neural network must be precisely tailored to the specific problem being solved.

- **Design Approach:** Topology design is empirical; there is no universal Standard Operating Procedure (SOP) that fits all datasets.
- **Node Allocation:**
  - Classification tasks require one output node (for binary) or multiple output nodes equal to the number of classes.
  - Regression tasks strictly require one output node.
- **Starvation vs. Saturation:**
  - **Starvation:** Occurs when a network possesses too many weights but receives insufficient training data to update them accurately.
  - **Saturation:** Occurs when a network receives massive amounts of data but lacks sufficient internal weights to model the complexity.

Unsupervised Learning (Clustering)

Algorithms designed to identify hidden patterns, relationships, or structures in unlabelled data without explicit external guidance.

K-Means Clustering

Developed by Stuart Lloyd in 1957, K-Means groups text documents into k distinct clusters based on content similarity.

- **Applications:** Topic modelling, large-scale document grouping, sentiment clustering, and language identification.
- **Mechanism:** Randomly places k initial centroids, assigns each document to the closest centroid, and iteratively recalculates the new centroid positions until the clusters stabilize.
- **Distance Metrics:** The distance is generally computed using standard Euclidean distance or normalized cosine distance.

Computing the Value of K

The scalar value of k dictates the precise number of clusters (or classification types) the algorithm will generate.

- **Heuristics:** Setting $k = 1$ merges all data into a single cluster; setting $k = |\text{Data Frame}|$ creates a dedicated cluster for every single point. A common baseline heuristic is $k = \frac{|\text{Data Frame}|}{2}$.
- **The Elbow Method:** A technique that involves incrementally increasing the value of k to measure the resulting Sum of Squared Errors (SSE).
- **SSE:** Represents the sum of the squared distances between a given centroid and all elements assigned to its specific cluster.
- **Inflection Point:** The optimal k (the "elbow") is the exact inflection point on the graph where increasing k further yields no significant mathematical reduction in the error rate.

Week 8: Introduction to Word Embeddings

Fundamentals of Word Embeddings

Word embeddings are dense, low-dimensional numerical representations of words. Unlike sparse methods (BOW, TF-IDF) that result in vectors the size of the vocabulary, embeddings represent words as fixed-size vectors of real numbers (typically 50 to 300 dimensions).

- **Semantic Encoding:** Captures the meaning of a word such that words used in similar contexts have similar vectors.
- **Similarity Computation:** Enables calculating similarity scores (e.g., Cosine Similarity) between words. It identifies not just synonyms, but also related concepts (e.g., "Galway" and "Ireland").
- **The 2013 Revolution:** The release of **Word2Vec** by Google transformed NLP by moving from discrete atomic symbols to continuous vector spaces.
- **Feature Representation:** A word embedding is a vector of numbers representing features of a word.
- **Fixed Vector Size:** Feature width is fixed (typically 50 to 300) and learned by a neural network.
- **Efficiency Gain:** Reduces comparison complexity from $O(n^2)$ to $O(n)$.
- **Storage:** Can be stored and queried efficiently in vector databases.
- **Latent Features:** Embedding dimensions capture hidden relationships that are not explicitly interpretable (unlike n-grams).

Static vs. Context-Sensitive Embeddings

- **Static Embeddings:** Every word has a single fixed vector regardless of context.
  - *Examples:* Word2Vec, GloVe, FastText.
  - *Limitation:* Cannot distinguish between polysemous words (e.g., "bank" of a river vs. "bank" for money).
- **Context-Sensitive Embeddings:** The vector for a word changes based on the surrounding words in a sentence.
  - *Examples:* BERT, ELMo, GPT.

Geometric Properties & Word Analogies

One of the most powerful features of word embeddings is that they capture linguistic relationships through vector arithmetic.

- **Analogy Logic:** The relationship between "Man" and "Woman" is geometrically similar to the relationship between "King" and "Queen".

- **Vector Arithmetic:**

$$V_{\text{King}} - V_{\text{Man}} + V_{\text{Woman}} \approx V_{\text{Queen}}$$

- **Distance Metrics:**
  - Cosine similarity (most common, range [-1, 1])
  - Euclidean distance
  - Manhattan distance
  - Hamming distance
  - Minkowski distance

One-Hot Encoding vs. Dense Embeddings

- **One-Hot Encoding:**
  - Vector size = Vocabulary size ($|V|$).
  - Categorical and sparse (mostly zeros).
  - **Weakness:** No notion of similarity. The dot product of any two distinct one-hot vectors is always 0.
- **Dense Embeddings:**
  - Vector size = Fixed hyperparameter (e.g., 100).
  - Continuous and dense.
  - **Strength:** Captures "closeness" in a high-dimensional manifold.

Word2Vec Architecture

Word2Vec is a predictive model that learns embeddings by trying to predict a word based on its neighbors (or vice versa).

- **Context Window:** A fixed-sized window of n words surrounding a target word.
- **Sliding Window:**
  - Moves across text to generate training data.
  - Produces (input, output) training pairs.
  - All words inside the window share the same context.
  - Window size controls context scope and number of training samples.
- **The Model:**
  - Input layer: One-hot encoded vector of size $|V|$.
  - Hidden layer: Linear projection (embedding layer).
  - Output layer: Softmax over vocabulary.
- **Weights as Embeddings:**
  - After training, the output layer is discarded.

  - The input $\rightarrow$ hidden weight matrix becomes the embedding lookup table.
- **Embedding Dimension:** Equal to the number of hidden layer units.
- **Scalability Issue:**
  - Weight matrix size = $|V| \times d$
  - Example: 31,218 words $\times$ 300 features $\approx$ 9.3 million weights
  - Makes training computationally expensive.

Week 9: Word2Vec Methods & Skip-Gram vs. CBOW

Continuous Bag of Words (CBOW)

The CBOW model predicts a **target word** based on its surrounding **context words**.

- **Goal:** Predict w_t given $\{w_{t-n}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+n}\}$.
- **Mechanism:**
  - Multiple context words are one-hot encoded.
  - Each maps to its embedding vector.
  - Embeddings are averaged into a single vector.
  - This vector is used to predict the target word.
- **Training Samples:**
  - Generates **one training sample per word**.
- **Efficiency:**
  - Faster to train than Skip-gram.
  - Works well for **frequent words**.
  - Captures **syntactic relationships** (e.g., drink, drinking, drinker).

Skip-Gram

The Skip-gram model does the inverse of CBOW: it uses a single **target word** to predict surrounding **context words**.

- **Goal:** Predict $\{w_{t-n}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+n}\}$ given w_t.
- **Training Samples:**
  - Generates **multiple samples per word**.
  - Number of samples $\propto$ window size.
- **Effectiveness:**
  - Better for **rare words**.
  - Works well with **small datasets**.
  - Produces more expressive embeddings.

- **Computation:**
  - More expensive than CBOW.

Mathematical Objective: Softmax and Cross-Entropy

The model aims to maximize the probability of the context word w_O given the center word w_I.

- **Softmax Function:**

$$P(w_O|w_I) = \frac{\exp(v'^{\top}_{w_O} v_{w_I})}{\sum_{w=1}^{V} \exp(v'^{\top}_{w} v_{w_I})}$$

- **Problem with Softmax:**
  - Requires computing probabilities over the entire vocabulary.
  - Time complexity: $O(V)$ per update.
  - Becomes infeasible for large vocabularies (millions of words).

Training Optimizations

To handle large vocabularies, Word2Vec uses approximation techniques:

Hierarchical Softmax

- Uses a binary (Huffman) tree.
- Reduces complexity from $O(V) \rightarrow O(\log V)$.

Negative Sampling (NEG)

- Replaces softmax with **sigmoid functions**.
- Converts problem into multiple binary classification tasks.
- Each output node acts as a **logistic regression classifier**.
- **Mechanism:**
  - Update:
    - 1 positive sample
    - ~5–20 negative samples
  - Only a small subset of weights updated per step.
- **Complexity:**
  - Reduced from $O(V) \rightarrow O(1)$ per update.

Subsampling

- Frequent words (e.g., “the”, “and”, “of”) are randomly discarded.
- **Motivation:**
  - These words generate too many redundant training samples.
- **Mechanism:**
  - Controlled by probability $P(w_i)$.
  - Typical threshold: 0.001.
- **Effect:**
  - Reduces training size
  - Speeds up training
  - Improves embedding quality

Context Position Weighting

- Words closer to the target word are more important.
- **Implementation:**
  - Randomly shrink the context window size.
  - Words near the center are more likely to remain.
  - Words further away are more likely to be dropped.
- **Result:**
  - Implicit weighting without explicit weights.

Skip-Gram vs. CBOW

- **CBOW:**
  - Faster and more efficient.
  - Produces fewer training samples.
  - Better for **frequent words**.
  - Better at **syntactic relationships**.
- **Skip-Gram:**
  - Slower but more expressive.
  - Generates more training samples.
  - Better for **rare words**.
  - Performs better on **small datasets**.
  - Larger window $\rightarrow$ more training data.

Week 10: Continued Word Embeddings & Transformer Architecture

Doc2Vec

- Extends Word2Vec to represent documents paragraphs or sentences as fixed-length vectors.
  - Adds a unique vector with document / paragraph id.
- Doc2Vec considers document membership as part of the context.
- Two main approaches:
  - PV-DM (Paragraph Vector - Distributed Memory): Similar to CBOW but with an additional document ID vector.
  - PV-DBOW (Paragraph Vector - Distributed BOW): Similar to Skip-gram.
- Captures the semantics and context of all words in all documents (all vs all).
  - Term distributed refers to the density of the vector.

Word2Vec Limitations

- While Word2Vec has been groundbreaking, it does have some limitations:
  - No context for polysemy: where a single word or phrase has multiple meanings. E.g. He has been drinking again or he is really wicked.
  - Scalability: Each word in a vocabulary requires a full vector. Very large dataset can be prohibitive.
  - Incomplete vocabulary: missing words in training data may cause inaccuracies in models.
  - Morphological words variations are coded independently, e.g. executable, execute, executed, execution and executioner semantics are not encoded.

Machine Learning with Embeddings

- Embeddings have many advantages over BOW and TF-IDF vectors as input vectors for ML models.
  - Capture semantic meaning. Similar words cluster together in vector space. Traditional methods are statistical and treat words as independent dimensions.
  - Create fixed-size dense vectors regardless of vocabulary size. More computationally efficient.
  - Enable analogical reasoning for mathematical operations, e.g. king - man + woman = queen.
  - Pre-trained models allow transfer learning, i.e. they can be applied to new tasks with limited data. BOW and TF-IDF must be recalculated for each new corpus.
- Embeddings capture semantics and context. They are also fixed length and can be used as ML inputs.

  - Semantics and context are encoded in dimensions. Commonly have 50 → 500+ dimensions / word.
- Vector space is $O(nd)$, where n = vocabulary size and d = vector dimension size. Vector length very large.
  - 1000 words of 300 dimensions ⇒ 300,000 inputs… Too large for efficient ML models. Need to squash vector.
- Lots of options for dimensionality reduction.
  - Can use simple aggregation methods, including domain-specific optimizations, to compress ML vector.
  - Can also use embeddings with BOW and TF-IDF.

Average Pooling

- Calculates a mean vector across all word embeddings in the document. Simple and effective.
- **Average Pooling Equation:**

$$\text{vector} = (\text{embedding}[d_1] + \text{embedding}[d_2] + \cdots + \text{embedding}[d_n])/n$$

  - Reduces space from $O(nd)$ to $O(d)$, where n = number of words in document and d = embeddings size.
- Preserves the general semantic meaning of the text.
  - Robust to document length variations and works well for many classification tasks. Loses word order information.
  - Gives equal importance to all words. Can dilute the signal from important but rare words.

Max Pooling

- Uses the maximum value found across all word embeddings for each dimension [i] in vector space.
  - Represents the most activated features and captures the most salient features across the document.
- **Max Pooling Equation:**

$$\text{vector}[i] = \max(\text{embedding}[i][d_1], \text{embedding}[i][d_2], \ldots, \text{embedding}[i][d_n])$$

  - Also reduces space complexity from $O(nd)$ to $O(d)$.
- Captures the strongest signals regardless of position.
  - Can identify important features even if they appear only once. Less affected by common words or padding.

  - Loses word order and feature frequency information. May overemphasize outlier words.

Transformer Architecture

- A neural network model that uses self-attention mechanisms to process sequential data efficiently.
  - An evolution of Seq2Seq encoder-decoder architecture.
  - Uses a self-attention mechanism to process input token relationships regardless of their position.
- The output of a transformer model is:
  - A prediction of the next token in a sequence based on all previous tokens. Generate a sequence of tokens.
  - The transformation an input sequence into a meaningful output sequence. Language translation.
- Replaces RNN Seq2Seq models like LSTM and GRU.
  - Lose context in long sequences. Vanishing gradient.

What is Attention?

- RNNs process tokens sequentially causing distant elements to be diluted or forgotten. Creates an implicit chain of computation.
- Transformers process tokens in parallel and create a matrix of relationships using attention scores.

Attention Scores

- Requires multi-step processing of tokens to create matrix. Multiple attention heads can focus on different relationships.
- A lie is a place in a river where salmon rest before moving on.

Transformer Model - Preprocess & Input Embeddings

- Input text is tokenized, converted to a vocabulary id and associated embeddings loaded.
- Need to represent positional information for each token to enable comprehension and parallelism.
  - Without it, a transformer cannot distinguish between “man bites dog” and “dog bites man”, i.e. we’ve a BOW.
  - Transformer would fail at understanding sequential tasks like translation and summarization.

- Positional encoding values are computed from varying frequencies of the sine and cosine functions.

Positional Encoding

- Positional encoding are computed just once for each index of the token embeddings.
  - Encoder Input = [token embeddings] + [positional embeddings]
- $\sin(0) = 0$ and $\cos(0) = 1$. For large dimensions sine values (even) approach 0 and cos values (odd) approach 1.
- Positional encoding uses sin() / cos() for even/odd indices. Sinusoidal patterns create unique encodings for each position.

Transformer Model - Encoder

- An encoder transforms the processed input embeddings into contextualized representations.
  - Outputs a set of vectors representing the input sequence with a rich contextual understanding.
  - Consists of a multi-headed attention mechanism and a feed-forward network. Whole encoder is a NN.
- Encoders are stacked. Original used a stack of 6.
  - Creates a deep network to model complex functions.
  - Each layer refines representations from previous layer.
  - Amplifies the portion of the input that influences a particular output position (Effective Receptive Field).
  - Helps capture long-range dependencies.

Self-Attention

- Allows the model to relate tokens to each other.
  - E.g. strongly relate flies, silver and lie to salmon.
- Need a way to convert the embeddings of dimension d for each token into this matrix. Done using Q, K and V matrices.

Scaled Dot Product Attention

- Scaled dot-product attention is at the heart of a transformer. Uses Q, K and V vectors (2D matrices).

  - Attention score is the dot product of Q and K vectors, scaled by $\sqrt{\text{dimension}}$ of the key vectors.
  - Determines how much each token should attend to every other token.

- The values in the QK^T matrix are called attention scores. Queries (Q) and Keys (K) that are similar will have a larger dot product. Scaling is needed to ensure a stable gradient during training. Matrix multiplication may generate large scores.

- Encoder inputs multiplied by weight matrices to produce QKV. Weights learned during training.

  - Q (Query): What each token is looking for: $Q = X \cdot W_q$
    * W_q learns to transform token embeddings into query vectors that ask questions of other tokens.
  - K (Key): What each token offers: $K = X \cdot W_k$
    * W_k learns to transform token embeddings into key vectors that answer questions from queries.
  - V (Value): Information contained by tokens: $V = X \cdot W_v$
    * W_v learns to transform token embeddings into value vectors that contain information to be aggregated.

- The original dimensions of the 2D matrix are restored, i.e. 4 x 5. The attention weights represent the All vs All contextual importance of the tokens. Logically the same as the weighted sum of the inputs from a hidden layer to the next layer of neurons in a MLP.

Multi-head Attention

- A key component of transformer architectures.

  - Slices the Q, K, V matrices and processes each slice with a separate head. 8 heads used in original.
  - Converts matrix of [number of tokens][dimension] into n matrices of size [number of tokens][dimension / n], where n = number of heads.

- Enables parallel learning of different relationships that a single attention matrix cannot capture alone.

  - Matrix transposition and multiplication means different attention scores will be learned with multiple heads.
  - Can focus simultaneously on different parts of the input.
  - Learns complex relationships by projecting input into multiple subspaces before computing attention.

- Concat does not add vectors. It joins them together to form a longer vector.

- Number of attention heads depends on the model architecture and size, but powers of 2 often used.

  - BERT base has 12 heads and BERT large has 16. GPT-3 had up to 96. Typically, 16 - 128 heads are used.

- More heads enable simultaneous attention to more input patterns. Increases computational complexity.
  - Can overlap heads to for redundancy / robustness.
- Slicing into 2^n sizes helps GPU parallelization.
  - GPUs more efficient with matrices / vector of 2^n. This includes GPU matrix multiplication libraries.
  - CUDA (Compute Unified Device Architecture) warps computation into groups of 32 threads in NVIDIA GPUs.

Add & Normalize Layer

- A residual connection in a transformer is a direct path that bypasses a sublayer, e.g. attention or FFNN.
  - Prevents original input being lost in stacked sublayers.
  - Mitigates vanishing gradient / reduces training time.
- Encoder adds original input, X, to attention(X) and then normalizes the values for the FFNN:
  - Output = LayerNorm(X + MultiHeadAttention(X))
- Normalization calculates the mean and variance of the features at each position and rescales the values.
- **Layer Normalization:**

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma + \epsilon} + \beta$$

  - μ is the mean at a position, σ is stddev, ϵ is a small constant 1e-5, γ and β are learnable parameters.

Feed-Forward Network

- Normalized attention vectors are the result of matrix multiplications and are linear transformations.
  - Non-linearity applied by transforming vectors using a dense FF neural network. FFNN is just another matrix.
- For the input vector x at each row the FFN computes:
- **FFN Equation:**

$$\text{FFN}(x) = W_2(\text{ReLU}(W_1 x + b_1)) + b_2$$

  - W_1: weights for the hidden layer. Usually 2048 nodes, i.e. a matrix of dimensions $\times$ 2048 values.

  - W_2: weight matrix for the second linear transformation (2048 $\times$ dimensions)
  - b_1 and b_2: bias vectors learned during training.
- Transformer power based on attention and FFNN.
- Information flows through the encoder once. It only generates one output.

Encoder Output

- The encoder output is a set of vectors representing the input with a rich contextual understanding.
  - A 2D matrix with dimensions len(tokens) $\times$ embeddings.
- Encoder output enough for classification tasks:
  - Sentiment analysis, topic modelling, spam detection.
  - Can use for labelling (PoS tagging) and similarity (IR).
- Encoder output can be fed as input to a ML model:
  - ML model called a classification head. Fine tune a pre-trained encoder (BERT) with new ML and labelled data.
  - Just use encoder as a fixed feature extractor without updating its weights. Train the new ML model only.
  - Can also postprocess encoder output using pooling etc.

Transformer Model - Decoder

- Takes an encoded input representation and outputs tokens one at a time using autoregressive generation.
  - Information flows through a decoder every time a new token needs to be generated. Decoders are stacked.
  - The output of the decoder is fed back into the decoder as the next input. Starts with [SOS] $\rightarrow$ [BOS] $\rightarrow$ [EOS].
- Most of a decoder components are identical to those in an encoder. The different components are:
  - Masked Multi-Head Self-Attention: prevents decoder looking at future tokens when generating a sequence.
  - Cross-Attention: allows the decoder to stay focused on the relevant parts of the encoder input.

Masked Multi-Head Attention

- Enables autoregressive behavior by preventing model "seeing into the future" during training / inference.

  - Each token can only attend to itself and previous tokens in the sequence. Called causal masked attention.
  - Causal respects the natural ordering of a sequence.
- The iteratively generated decoder sequence is preprocessed and an attention mask applied.
  - Masking done by setting future positions in attention matrix to $-\infty$ to before softmax. $\text{softmax}(-\infty) = 0$.
  - Ensures autoregressive generation. When predicting tokens at position i, decoder only sees tokens at positions $<$ i. All cells above the diagonal are zeroed out.
- Applying an attention mask means the decoder cannot look ahead. Values are zeroed out.

Cross Attention

- The encoder output serves as the memory that the decoder attends to. Helps prevent hallucinations.
  - Multiheaded attention applied to output of masked self-attention layer and encoder output.
  - Works exactly like the encoder attention layer but:
    - Query (Q): current causal decoder sequence.
    - Key / Value (KV): encoder output matrix.
- Prevents drift from the original prompt intention.
  - A bridge that connects the iterative generation of decoder output to the encoder representation of input.
  - Without cross attention, a LLM would be conditioned only on its own previously generated tokens.

Linear Layer & Softmax

- The decoder output (hidden states) is converted by a linear projection into logits over a whole vocabulary.
  - Softmax then applied to logits to generate next token.
- Final layer called language modelling or LM head.
  - Transforms high-dimensional representations into useful language predictions.
- Linear project is a matrix multiplication of:
- **Linear Project Equation:**

$$\text{logits} = \text{decoderout} \times W + b$$

  - decoderout: decoder output (tokens $\times$ dimensions).
  - W: a weight matrix of shape (dimensions $\times$ vocab size).
  - b: A optional bias vector of length vocab size.

- BERT / DistilBERT have a vocabulary size of 30,522 WordPiece tokens. GPT-4: 100,000 BPE tokens.
  - Claude: ~50,000 BPEs tokens. Gemini: ~256,000 SentencePiece BPEs. LLaMA2: 32,000 BPEs.

Week 11: Information Retrieval & Text Search

Information Retrieval (IR)

- Techniques for efficiently locating and ranking relevant documents within large-scale text collections.
  - Challenge: given a corpus of n docs and a query q, identify and rank all docs by relevance in $< O(n)$ time.
  - IR is foundational to managing the information explosion.
  - Enables effective access to data far beyond human browsing capacity. Vannevar Bush (1945).
- Combines data structures, algorithms, linear algebra, graph theory and distributed systems.
  - Use optimized inverted indexes, PageRank and link analysis, web spam detection and web crawling.
  - Technologies used by Google, Apache Lucene etc.

Inverted Index Data Structure

- The fundamental data structure in IR and the backbone of how search engines work efficiently.
  - Used by virtually all search engines, including Google, Elasticsearch, Apache Lucene and Apache Solr.
- Inverts the relationship between documents and words / tokens. Like an index at the back of a book.
- In practice, the tokens will be represented by integers. Fixed length 32 bits vs strings.
- Transforms a search of all documents into a dictionary look-up and processing of a postings list.
  - Implemented as a mapping of $f : T \to P$, where T is a term and P is a postings list. A sorted list of DocIDs.
- Requires very fast CRUD operations on n indexed documents. Must support concurrency.
  - Dictionary / vocabulary lookup can be done in $O(\log n)$ or $O(1)$. Hash Map, Skip List, B-Tree, Radix (Prefix) Tree.
  - Posting list of document IDs and positions processed in $O(n + k)$, where k is the size of the positions index.
  - Use Array / Compressed Array, Skip List, Bitmap / Bit Vector, Roaring Bitmap. Usually compressed data.

- The key is a term, a normalized word or token. The value is the postings list for that term, an ordered list of documentIDs where the term appears. The positions list is sorted. This is critical for rapid search and query / set operations.

Inverted Index Query Operations

- Boolean logic maps directly to efficient set operations on sorted postings lists in an inverted index.
  - Enables efficient AND, OR, NOT and phrase queries.
  - Time complexity depends on data structure used.
- Single-Term Query (Set Retrieval)
  - Retrieve all documents containing a specific term., e.g. Query algorithm → {doc1, doc5, doc7, doc12}.
  - $O(1)$ lookup + $O(k)$ where k = posting list length.
- AND Query (Intersection)
  - Returns documents containing all query terms, e.g. graph AND algorithm. Process shortest list first.
  - $O(m + n)$ where m, n are posting list lengths.
- OR Query (Union)
  - Returns documents containing any query term, e.g. graph OR algorithm. $O(m + n)$ complexity.
- NOT Query (Complement/Difference)
  - Returns documents not containing a term. Set difference or complement query, e.g. NOT algorithm. $O(d)$ complexity where d is the total number of documents.
- Phrase Query (Positional Intersection)
  - Returns documents containing an exact phrase (term sequence) , e.g. data structures and algorithms.
  - Requires a sorted positional index. $O(m + n + p \cdot q)$ where p, $q \rightarrow$ average positions of each term in m, n.

Inverted Index Optimisation

- Optimisations also include query processing (Skip Lists, Pointers, Galloping Search) and caching lists.
- **Variable-Byte Encoding (VByte)**
  - A compression technique that encodes integers with a variable number of bytes based on their magnitude.
  - Smaller numbers use fewer bytes. Instead of using 4 bytes (32 bits) per integer, chain together 7-bit chunks.

- **Delta Encoding**
  - Compresses by storing differences (deltas) between consecutive values instead of the values themselves.
  - Fundamental for scaling search engines. Otherwise indexes would consume prohibitive amounts of storage.
  - Scalability issues using 32-bit ints in postings lists.
  - Instead of storing a sorted list of document IDs, store a list of the VByte-encoded deltas:
    * term: "algorithm"
    * postings: [101, 105, 107, 112, 150, 151, 152, …]
    * deltas: [101, 4, 2, 5, 38, 1, 1]
  - Most gaps are small numbers that can be encoded using far fewer bits (variable-length encoding).
  - Typically reduces storage for postings list by 50-80%.
- **Bit Packing**
  - Compresses data by packing an integer into the smallest number of bits possible.
  - Leading zeros are a waste of space! Can compact multiple smaller values into 32-bit ints or 64-bit longs.
  - Can use variable-length encoding to maximise compression! Bit size given as metadata in header.

Compressed Inverted Index

- Compression significantly reduces the index size.
- Needs to be balanced against the overhead of decoding the compressed index.

Spam and Search Engines

- Computing and ranking search results from terms in an inverted index can easily fall prey to spam.
  - Term results from an inverted index are scored and ranked by an algorithm. Links examined too.
- Search engine rankings affected by term and link spam. Search engine "optimisation"…
  - Term spam: convincing a search engine that a page represents something it is not. Done by adding terms.
  - Add lots of terms, copy #1 search result into page.
  - Link Spam: generate lots of spurious back links.
  - Link farms, reciprocal linking schemes, Comment / forum spam, hidden links, expired domain hijacking.

Link-Based Search

- Analyses the hyperlink structure of a document graph to determine page importance and authority.
  - Not just content and term frequency / weights.
  - Link structure is content-independent.
- Not all pages are equal. Links work like citations.
  - A link from an authoritative page (IEEE / Gov) carries more weight than a link from a random blog.
- Highly effective against term spam:
  - External and Costly: Difficult and costly to manipulate backlinks from authoritative sites.
  - Trust: Spam sites tend to form isolated clusters with links only among themselves. Detect with link analysis.

Early Search Engines

- **Lycos Search Engine - 1994**
  - Based on the Pursuit retrieval engine developed by Michael Mauldin at Carnegie Mellon in 1994.
  - Inverted index with a vector space model. First search engine to scale massively (60M pages by 1996).
  - Fast query processing. Partial matches with prefixes.
  - No link analysis (web structure) or notion of document authority. Susceptible to keyword stuffing / spamming.
- **AltaVista Search Engine - 1996**
  - Developed by DEC, AltaVista was the preeminent internet search engine before Google.
  - Ranked document content using vector space scoring and cosine similarity. Used HITs to analyse links.
  - Vector space can measure content similarity using cosine distance. HITS algorithm measures relevance based on link indegree and outdegree.

Hyperlink-Induced Topic Search (HITS)

- HITS is a link-based ranking algorithm that models web pages in two complementary roles:
  - Authorities: pages that are linked to by many hubs, e.g. a course page, a W3C specification.
  - Hubs: pages that link to many authoritative pages, e.g. a wiki list of computing degrees, a list of ML APIs.
- HITS assigns two scores to each page, creating a mutually-reinforcing relationship.

  - A good hub points to good authorities. A good authority is pointed to by good hubs.
  - Creates a positive feedback loop that amplifies relevant pages. Score using Principle of Repeated Improvement.
- The time complexity is $O(k \cdot (|V| + |E|))$, as each iteration processes all vertices and edges once.
  - A value of $k = [20..30]$ is typically needed for convergence. Efficient for sparse graphs common on the web.

Limitations of HITs

- HITs is query-dependent and semantically aware, with auth scores computed for each query.
  - Identifies pages that are genuinely authoritative within that semantic domain rather than globally popular.
- But cost per query is too expensive at scale:
  - Identifying a topic-specific subgraph.
  - Expand subgraph to include neighbourhood pages.
  - Execute 10-30 iterative computations for each query.
- HITs is vulnerable to topic drift and link spam.
  - A spammer only needs to create hubs that link to their own authority pages to artificially inflate HITs scores.

PageRank

- Original Google search algorithm. Developed at Stanford in 1996 by Larry Page and Sergey Brin.
  - Their BackRub search engine analysed back links pointing to a website. Similar to an academic citation.
- PageRank is modelled as a random surfer on the web, randomly clicking hyperlinks.
  - At each page, they follow an outgoing link with probability d, a damping factor, typically 0.85.
  - With probability $1 - d$, they teleport to a random page, representing getting bored and typing a new URL.
  - The PageRank of a page is the probability the surfer is on that page at any given time after n steps.
- The importance of a page is proportional to the number and quality of pages that link to it.
  - The damping factor of 0.85 is based on empirical data. 85% of the time, a web user will follow a link on the existing page.
- The time complexity is $O(k \cdot (|V| + |E|))$, as each iteration processes all vertices and edges once.

TrustRank

- TrustRank uses a small set of trustworthy seed sites.
  - Propagates trust backward from seeds through the link graph. Seeds are authoritative, e.g. educational.
- PageRank assumes that link structure reflects quality.
  - High-authority pages link to other high-authority pages.
  - Treats all links equally and iteratively distributes authority globally. Breaks down with link spam / spam farms.
- PageRank vs TrustRank:
  - PageRank treats all links equally. Can manipulate backlinks to boost ranking. (e.g. link farms, purchased links).
  - TrustRank: Trust cumulatively boosts the ranking from specific authoritative seeds.

Week 12: Sentiment Analysis

Sentiment Analysis Fundamentals

- The process of determining the emotional tone or attitude expressed in a piece of text or speech.
  - Overall emotion is classified in terms of polarity as being positive, neutral or negative. Range is [1, 0, -1].
  - The term valency is the emotional direction and intensity of a word or phrase. Range is [-5…+5].
  - Used to extract subjective information from a source.
- Has wide applicability to situations where opinions can be mapped to real world events.
  - Feedback analysis for service / product improvement.
  - Monitoring social media for perception / opinion.
  - Financial market predictions using news sentiment.

Evaluation Methods

- Main approaches involve lexicons and ML models.
  - Hybrid and transformer approaches also used.
- Lexicons: a static dictionary of words or phrases and their associated sentiment scores. Very fast.
  - Often manually curated. Different scoring systems.

- ML involves feature engineering training, test and production text using techniques like:
  - Traditional: BoW, TF-IDF, n-grams.
  - Word Embeddings: Create a document vector by averaging / pooling the embeddings for each word.
  - Contextual Embeddings: use the hidden states output from a transformer encoder. Join or pool vectors.

Key Lexicons

- **Bing Liu's Lexicon:**
  - Widely used simple lexicon of 6,800 opinion words. Contains ~2,000 positive and ~4,800 negative words.
  - No scores associated with words, but fine for basic sentiment analysis. English only.
- **AFINN Lexicon:**
  - A word based lexicon by Finn Årup Nielsen. Words rated from -5 (extremely negative) to +5 (extremely positive). AFINN provides intensity levels. Different languages.
- **TextBlob (Pattern) Lexicon:**
  - A Python library for processing textual data that includes its own lexicon. Polarity: [-1..1].
  - Modified Pattern Lexicon. Includes phrases and provides subjectivity scores ($0 \rightarrow$ objective, $-1/1 \rightarrow$ subjective).
- **VADER Lexicon:**
  - Valence Aware Dictionary and sEntiment Reasoner.
  - Circa 7,500 words rated from -4 (extremely negative) to +4 (extremely positive). Human validated scores.
  - Special handling for social media type sentiments: Emoticons (e.g. :)), Acronyms (e.g., LOL), Slang (e.g. meh, nah).
  - Very effective for short texts, e.g. tweets, social media posts and short reviews where context may be limited.
- **MPQA Subjectivity Lexicon:**
  - Multi-Perspective Question Answering categorizes words and phrases based on subjectivity and polarity.
  - Manually curated lexicon of 8,000 words and phrases tagged with subjectivity information.
  - Words have a Subjectivity type (strongsubj / weaksubj), a Polarity (positive, negative or neutral) and a PoS.
- **SentiWordNet Lexicon:**
  - A lexicon based on WordNet with 117,000+ words. Sentiment annotation for English nouns, verbs, adjectives and adverbs.

  - Sense distinction and has three sentiment scores: Positivity: [0..1], Negativity: [0..1] and Objectivity.
  - **Objectivity Equation:**

$$\text{Objectivity} = 1 - (\text{Positivity} + \text{Negativity})$$

  - Performs well for formal, edited text but can lags behind Vader for social media content.

- **NRC Word-Emotion Lexicon:**
  - Words and their associations with eight basic emotions and two sentiments. Multiple languages.
  - Emotions: anger, fear, anticipation, trust, surprise, sadness, joy, and disgust. Sentiments: neg or pos.

Scoring Mechanisms

- **Basic Sentiment Scoring:**
  - Assigns numerical or categorical values to text based on the emotional tone it conveys. Treats all elements equally without applying weights.
  - Fast, transparent, easy to implement / adopt to different languages. Good for preliminary processing.
  - Cannot handle context, negations, sarcasm or irony.
- **Weighted Sentiment Scoring:**
  - Computes a sentiment score by prioritizing / weighting customizable factors of words or phrases.
  - Weights can be based on word position, PoS importance (adjectives weighted higher) and TF-IDF scores.
  - Customizable and produces more nuanced results than simple averaging but domain dependent.
  - Weights can handle negations (not) / intensifiers (very).

VADER Specific Properties

- Vader combines a lexicon of words with grammatical and syntactical rules to determine sentiment scores.
- The document score then normalized [-1..1] using a sigmoid function. Individual word scores are [-5..5].
  - score $\geq 0.05 \Rightarrow$ Positive
  - score $\leq -0.05 \Rightarrow$ Negative
  - $-0.05 <$ score $< 0.05 \Rightarrow$ Neutral
- Vader also reports the overall positive, negative and neutral sentiment scores for a document.

  - Human-validated lexicon and can handle negations, intensifiers, punctuation emphasis (!), capitalization (ALL CAPS) and contrastive conjunctions ("but", "however").

Distance-Weighted Negation

- Accounts for the impact of negation words based on their distance from sentiment-bearing terms (e.g. Vader).
- Captures how negation diminishes with distance:
  - Negation words closer to sentiment terms have a stronger effect than those farther away.
  - Denominator ensures that as distance increases, the negation effect decreases proportionally.